

MAI 5301 - Week 11

Presentation

Daryl Nelson
USI: 1021215

Background

Large language models (LLMs) have evolved from being impressive text generators into powerful interactive agents capable of reasoning, planning, and acting in complex environments. A major challenge in this transition is enabling LLMs to learn from experience and improve over time, rather than simply responding to prompts. The ReAct framework was an early step toward this goal, showing that LLMs can be structured to reason and act together not just think step-by-step, but also interact with external environments such as APIs or interactive environments, bridging the gap between internal reasoning and practical action. This approach improved performance on tasks like question answering and interactive decision making, while making the model's behavior more interpretable by combining thought traces with actions.

Building on this idea of language models as agents, Reflexion takes a further leap by teaching models to learn from their mistakes in a humanlike way through self-reflection, not weight adjustments. Instead of requiring costly reinforcement learning retraining, Reflexion enables agents to analyze feedback linguistically, store verbal reflections, and use them as a form of experiential memory to guide future attempts across tasks ranging from complex reasoning to code generation. This shift from experiment-only methods to iterative, reflective learning is a step toward agents that can adapt and improve autonomously in real-world settings, such as conversational assistants, automated problem solvers, and self-improving software tools. Through these innovations, both works contribute foundational steps toward making LLMs not just powerful, but practically useful intelligent systems that can reason, act, and learn over time with minimal human supervision.

SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao*,¹, Jeffrey Zhao², Dian Yu², Nan Du², Izhak Shafran², Karthik Narasimhan¹,
Yuan Cao² ¹Department of Computer Science, Princeton University ²Google Research, Brain
team

Motivation: Why Combine Reasoning & Acting?

Human Intelligence

- Humans naturally combine *thinking* and *doing*
- Large language models (LLMs) are good at reasoning or acting separately, but rarely both together.
- “Inner speech” helps with:
 - Planning
 - Self-correction
 - Memory tracking
 - Adapting to new situations

Problem in AI

- Chain-of-Thought (CoT) → reasons internally only
 - Can hallucinate
 - Cannot update with new information
- Action-only agents → act in environments
 - Limited high-level reasoning
 - Weak long-term planning

Key Gap: Reasoning and acting are treated separately.

ReAct: Reason + Act

Core Idea: Interleave reasoning steps and actions.

Model alternates between:

- Thought → Action → Observation → Thought → Action...

Benefits

- *Reason to act* → Plan better actions
- *Act to reason* → Gather new external information
- Dynamic plan updates
- Reduced hallucination
- Better decision-making

Results & Contributions

Evaluated On:

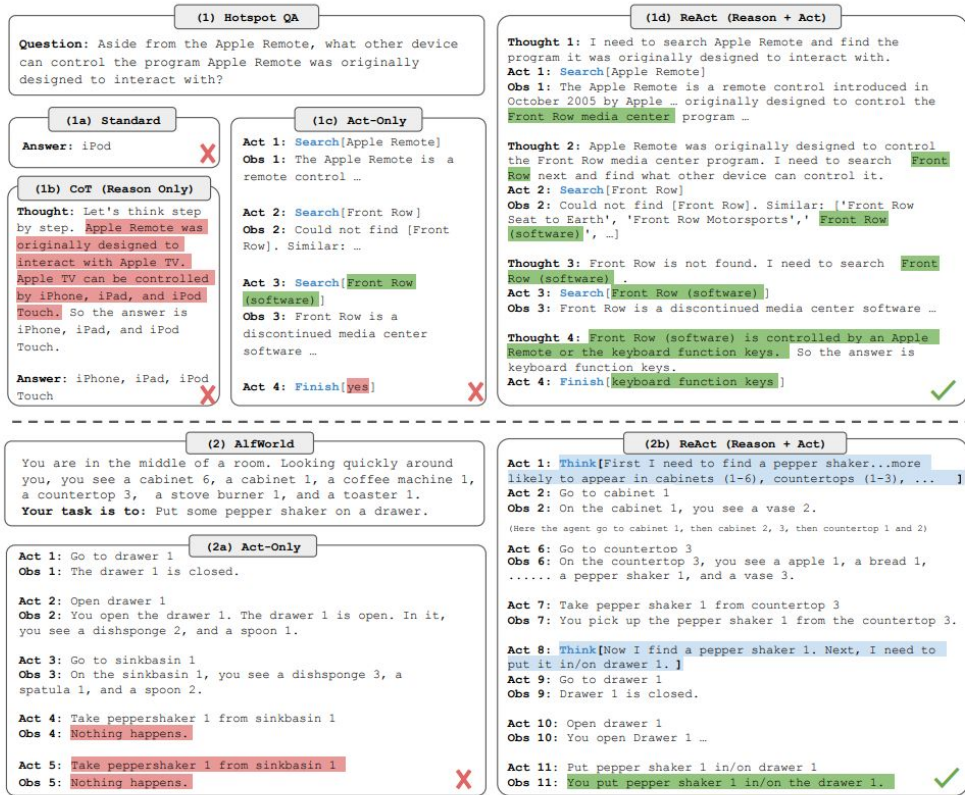
- Question Answering (HotPotQA)
- Fact Verification (FEVER)
- Text-based Game (ALFWorld)
- Web Navigation (WebShop)

Key Findings

- Outperforms action-only models
- Competitive with Chain-of-Thought
- Combining ReAct + CoT works best
- Large gains in interactive tasks (up to +34%)

Main Contributions

1. New Reason+Act prompting framework
2. Strong few-shot performance
3. Better interpretability
4. Shows reasoning + acting work best together



Problem Setup

Agent–Environment Interaction

- Agent receives observation (o_t)
- Chooses action (a_t) based on past context
- Context = previous observations + actions

Challenge

- Correct action often requires complex reasoning
- Action-only agents:
 - Fail in multi-step reasoning (QA)
 - Repeat incorrect or hallucinated actions
 - Struggle with long-term planning

Core Issue: Mapping context $\rightarrow$ action is too implicit without reasoning.

Mathematical Insight

Observation

$$o_t \in \mathcal{O}$$

- At time t , the agent receives information from the environment.
- Example: a question, a webpage result, or the current state.

Think of this as: *“What do I see right now?”*

Context (History)

$$c_t = (o_1, a_1, o_2, a_2, \dots, o_t)$$

- This is the full history up to time t .
- It includes:
 - All past observations
 - All past actions

Think of this as: *“What has happened so far?”*

Mathematical Insight

Policy

$$\pi(a_t \mid c_t)$$

- This means:
 - The agent chooses action at
 - Based on the full history c_t

Decision = function of everything I know so far

In a Normal Agent - Action Space

$$a_t \in \mathcal{A}$$

- $\mathcal{A}$ = real actions in the world.
- When the agent acts:
 - The world changes
 - A new observation appears

Loop: Observation $\rightarrow$ Action $\rightarrow$ Observation $\rightarrow$ Action

Mathematical Insight

What ReAct Changes - ReAct expands what the agent is allowed to do.

New Action Space

$$\hat{\mathcal{A}} = \mathcal{A} \cup \mathcal{L}$$

This means:

- $\mathcal{A}$ = real world actions
- $\mathcal{L}$ = language “thought” actions

Now the agent can either:

- Act in the world
- OR think first

Mathematical Insight

What is thought - Thought Action

$$\hat{a}_t \in \mathcal{L}$$

A thought:

- Does NOT change the world
- Produces NO new observation
- Only updates internal history

Updating the Context After Thinking

If the agent thinks:

$$c_{t+1} = (c_t, \hat{a}_t)$$

This means:

- The new thought is added to the history.
- The agent now has more structured reasoning for the next decision.

Thinking improves the next action.

Mathematical Insight

Standard Agent:

$$\pi(a_t \mid c_t), \quad a_t \in \mathcal{A}$$

ReAct Agent:

$$\pi(a_t \mid c_t), \quad a_t \in \mathcal{A} \cup \mathcal{L}$$

The only difference: ReAct allows thinking steps inside the action space.

ReAct: Expanding the Action Space

Idea: Combine thinking with acting in one framework.

New Action Space:

- External Actions (A): Interact with the environment (e.g., search Wikipedia, click, move).
- Language Thoughts (L): Internal reasoning – do not change the environment.

What Thoughts Do:

- Break down complex goals → subgoals
- Pull out important information from observations
- Add commonsense knowledge to guide decisions
- Track progress and adjust plans

Workflow: Thought → Action → Observation → Thought → ...

Why ReAct Works

Implementation

Uses a large pretrained language model (PaLM-540B)

Learns from few examples of human-written task trajectories

Thoughts (reasoning steps):

- Frequent for reasoning-heavy tasks
- Sparse for long decision-making tasks

Advantages

- Easy to design prompts — just write down thoughts alongside actions
- Works on many types of tasks (QA, games, web navigation...)
- Strong performance even with very few examples
- Transparent & controllable — humans can see and adjust reasoning steps

Takeaway: ReAct combines reasoning + acting in a single, flexible framework

Knowledge-Intensive Reasoning Tasks

Goal: Test ReAct on tasks that need both reasoning and information retrieval.

Datasets:

- HotPotQA: Answer questions that require connecting info from multiple Wikipedia pages.
- FEVER: Check if a claim is SUPPORTED, REFUTED, or if there's NOT ENOUGH INFO.

Setup:

- Model only sees the question or claim.
- No extra paragraphs provided.
- Must either:
 - Use its own knowledge
 - Or search Wikipedia to find the answer

Action Space & ReAct Prompting

Wikipedia API (Simple & Limited)

- `search[entity]` → first 5 sentences
- `lookup[string]` → find specific info
- `finish[answer]` → submit answer

(Weaker than modern retrievers → forces explicit reasoning)

ReAct Prompting

- Few-shot human-written trajectories
- Interleaved:
 - Thought → Action → Observation
- Thoughts used to:
 - Decompose question
 - Extract key facts
 - Apply common sense/arithmetic
 - Reformulate searches
 - Synthesize final answer

Baselines Compared

- Standard
 - No reasoning, no retrieval
 - Simply predicts answers from the question
- Chain-of-Thought (CoT)
 - Only reasoning, no external knowledge retrieval
 - Builds step-by-step thought process internally
- CoT + Self-Consistency (CoT-SC)
 - Samples multiple CoT reasoning paths
 - Picks majority answer to reduce errors
- Act-only
 - Only performs retrieval actions
 - No reasoning about what or how to use the information

Prompt Method^a	HotpotQA (EM)	Fever (Acc)
Standard	28.7	57.1
CoT (Wei et al., 2022)	29.4	56.3
CoT-SC (Wang et al., 2022a)	33.4	60.4
Act	25.7	58.9
ReAct	27.4	60.9
CoT-SC $\rightarrow$ ReAct	34.2	64.6
ReAct $\rightarrow$ CoT-SC	35.1	62.0
Supervised SoTA^b	67.5	89.5

Table 1: PaLM-540B prompting results on HotpotQA and Fever.

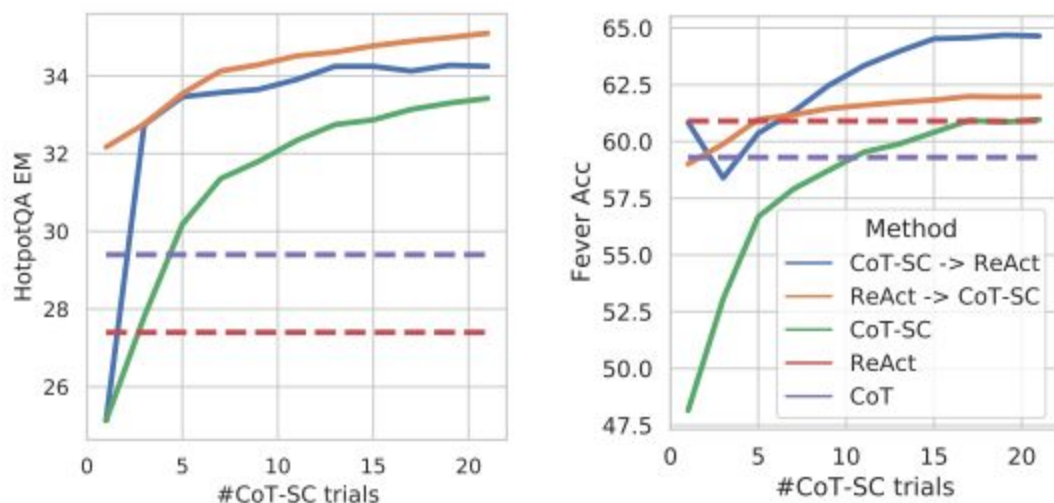


Figure 2: PaLM-540B prompting results with respect to number of CoT-SC samples used.

Key Results & Insights

ReAct vs Act-Only

- ReAct consistently outperforms Act-only
 - Reasoning helps guide what to retrieve and improves final answers
 - Example: On HotPotQA & FEVER, ReAct achieves ~60.9% vs Act ~50% accuracy

ReAct vs CoT (Chain-of-Thought)

- ReAct:
 - More factual & grounded thanks to external knowledge access
 - Lower hallucination: 6% false positives vs CoT 14%
- CoT:
 - More flexible reasoning steps
 - Higher hallucination risk: major failure mode 56%

Hybrid Approach Works Best

- ReAct + CoT-SC outperforms both individually
 - Combines internal knowledge (CoT-SC) + external retrieval (ReAct)
 - Reaches CoT-SC performance with only 3–5 samples instead of 21

Fine-Tuning Results

Fine-Tuning Results

Prompting Only

- Smaller models struggle with ReAct
- Hard to learn both reasoning and acting from few examples

After Fine-Tuning (3,000 trajectories)

- ReAct becomes the best method
- Smaller fine-tuned models outperform larger models using only prompting

Why it works

- Teaches the model how to retrieve information effectively
- Teaches dynamic reasoning over tasks
- More generalizable than just memorizing facts

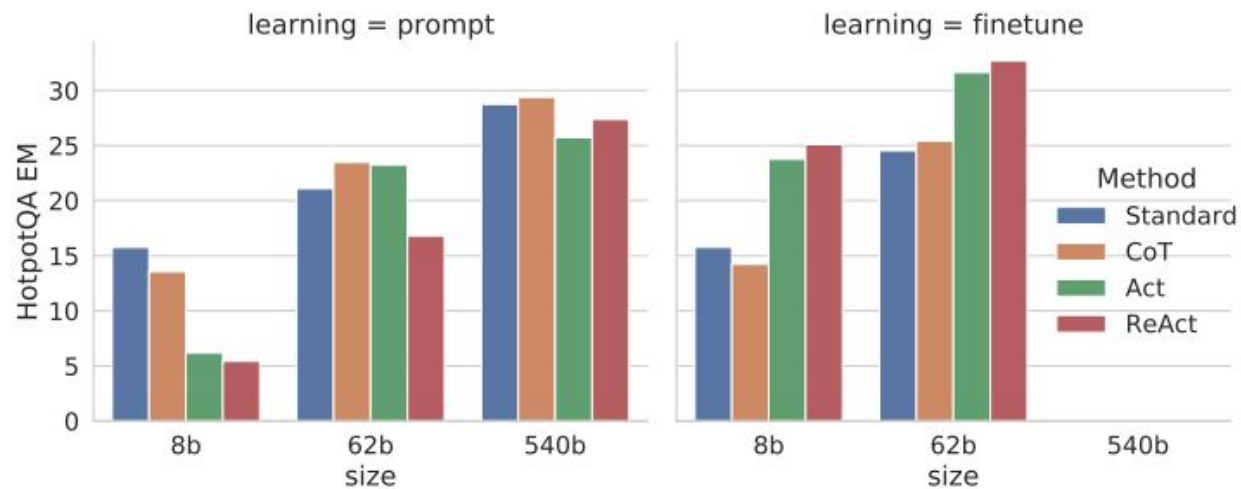


Figure 3: Scaling results for prompting and finetuning on HotPotQA with ReAct (ours) and baselines.

Decision-Making Tasks

Goal:

- Test ReAct in interactive environments where the model must plan over many steps

Key Challenges:

- Long sequences of actions
- Rewards are rare (sparse feedback)
- Need to track progress carefully
- Must explore strategically
- Requires commonsense reasoning

Environments Used:

- **ALFWorld:** Text-based household simulation (finding objects, completing tasks)
- **WebShop:** Online shopping task with real product options and user instructions

ALFWorld

What is it?

- A text-based simulation of a household
- Agent interacts with objects using text commands

Example Goal:

- “Put paper under the desk lamp”

Why is it hard?

- 50+ locations to explore
- Tasks can take 50+ steps
- Agent must:
 - Break big goals into smaller subgoals
 - Keep track of progress
 - Use commonsense knowledge (e.g., lamps are usually on desks or tables)

ALFWorld

Baselines

- Act-only prompting – retrieval without reasoning
- BUTLER – imitation learning agent trained on many expert examples

Result

- ReAct: 71% success
- Act-only: 45%
- BUTLER: 37%

Reasoning helps plan & stay on track.

Method	Pick	Clean	Heat	Cool	Look	Pick 2	All
Act (best of 6)	88	42	74	67	72	41	45
ReAct (avg)	65	39	83	76	55	24	57
ReAct (best of 6)	92	58	96	86	78	41	71
ReAct-IM (avg)	55	59	60	55	23	24	48
ReAct-IM (best of 6)	62	68	87	57	39	33	53
BUTLER _g (best of 8)	33	26	70	76	17	12	22
BUTLER (best of 8)	46	39	74	100	22	24	37

Table 3: AlfWorld task-specific success rates (%). BUTLER and BUTLER_g results are from Table 4 of Shridhar et al. (2020b). All methods use greedy decoding, except that BUTLER uses beam search.

Method	Score	SR
Act	62.3	30.1
ReAct	66.6	40.0
IL	59.9	29.1
IL+RL	62.4	28.7
Human Expert	82.1	59.6

Table 4: Score and success rate (SR) on Webshop. IL/IL+RL taken from Yao et al. (2022).

WebShop

What is it?

- Simulated online shopping environment
- 1M+ real products with structured & unstructured info
- Example task: *“Find a nightstand, nickel finish, under \$140”*

Why it's hard:

- Product info is noisy & inconsistent
- Requires multi-step actions: search, filter, compare
- Must decide the right moment to make a purchase

WebShop

Baselines:

- Imitation Learning (IL): Learns from human examples
- IL + Reinforcement Learning (IL+RL): IL plus trial-and-error improvement

Results:

- ReAct: +10% higher success than IL/IL+RL
- Better at picking products that match instructions
- Still not as good as humans

Why Reasoning Matters

Compared to:

- Action-only agents
- Inner Monologue (IM) style reasoning

Findings:

- Sparse, high-level reasoning works best
- Helps the model:
 - Break goals into smaller steps
 - Track which subgoals are done
 - Use common sense to guide actions
 - Decide what to do next

Key Takeaway:

- “Think before you act” → much better at long, multi-step tasks

Related Work: Reasoning with LLMs

Chain-of-Thought (CoT)

- LLMs can "think step by step" before answering
- Variants: Zero-shot CoT, Least-to-Most, Self-Consistency
- Improves multi-step reasoning accuracy

Other reasoning methods

- Selection-Inference: split reasoning into stages
- STaR: fine-tune on correct model-generated rationales
- Scratchpad: train on intermediate computation steps

Key point: Most methods focus only on internal reasoning, not on acting in the world.

Related Work: Decision Making with LLMs

Interactive decision-making

- WebGPT: browse web & answer questions, relies on human feedback
- Chatbots (BlenderBot, Sparrow): decide API calls, need large datasets

Robotics / Embodied AI

- SayCan: LLM suggests robot actions, filtered by physical feasibility
- Inner Monologue: adds environment feedback, but limited reasoning

ReAct Advantage

- Combines reasoning + acting
- Flexible, high-level planning & commonsense reasoning
- Cheaper learning via prompting instead of expensive datasets

Conclusion

ReAct Overview

- Synergizes reasoning and acting in LLMs
- Works across QA, fact verification, interactive decision-making
- Provides interpretable reasoning traces

Strengths

- Better performance than reasoning-only or acting-only methods
- Makes LLM decisions understandable and controllable

Limitations & Future Work

- Complex tasks need more examples → input length limits
- Fine-tuning improves performance
- Scaling + combining with reinforcement learning can unlock more potential

Takeaway: ReAct demonstrates that thinking and acting together leads to smarter, more trustworthy AI.

Q&A

Reflexion: Language Agents with Verbal Reinforcement Learning

Noah Shinn Northeastern University noahshinn024@gmail.com Federico Cassano Northeastern University
cassano.f@northeastern.edu Edward Berman Northeastern University berman.ed@northeastern.edu Ashwin Gopinath
Massachusetts Institute of Technology agopi@mit.edu Karthik Narasimhan Princeton University karthikn@princeton.edu Shunyu Yao
Princeton University shunyuy@princeton.edu

Introduction – The Challenge

Teaching AI Without Retraining

Main Points:

- Large language models (LLMs) like GPT can act as agents: play games, write code, use APIs.
- Traditional learning methods (like reinforcement learning) are slow and expensive.
 - Needs lots of trial-and-error
 - Requires retraining the model
- Goal: Make AI learn faster and smarter without changing its internal weights.

Visual Idea:

- Image of a student trying multiple times to solve a problem, looking frustrated.
- Tiny AI robot next to a huge weight labeled “Reinforcement Learning” vs a small book labeled “Reflection.”

Introduction - Learning by Reflection

AI Thinks About Its Mistakes

Main Points:

- Instead of retraining, the AI reflects in words:
 - “I failed because I forgot X. Next time I’ll do Y.”
- Reflections are stored in a memory buffer for future tasks.
- Feedback can be:
 - Numbers (score)
 - Text (comments, hints)
 - Self-generated (AI tests itself)

Analogy:

- Like a student keeping a diary of mistakes and lessons to improve next time.

Visual Idea:

- Flow: Attempt → Feedback → Reflection → Memory → Next Attempt → Success

How Reflexion Works

AI Learns Like a Human

Main Points:

1. AI performs a task (coding, reasoning, or decision-making).
2. AI receives feedback: “Success/Fail” or detailed hints.
3. AI converts feedback into textual reflections.
4. Reflections are stored in episodic memory.
5. Next attempt: AI reads reflections → improves its decisions.

Advantages over traditional RL:

- No retraining needed → lightweight
- Feedback is more detailed → nuanced improvement
- Memory is interpretable → we can see what AI learned

Visual Idea:

- Diagram of a circular loop with arrows: Task → Feedback → Reflection → Memory → Next Task

Introduction - Results & Impact

Reflexion in Action

Main Points:

- Reflexion improves AI in three main areas:
 1. Decision-making: +22% success in sequential tasks
 2. Reasoning: +20% improvement in knowledge tasks
 3. Coding: +11% improvement on Python problems
- Outperforms previous models like GPT-4 on coding benchmarks (91% vs 80%).
- Key takeaway: AI can learn faster and smarter just by reflecting, without heavy retraining.

Visual Idea:

- Bar chart showing Reflexion vs GPT-4 performance across tasks.
- Illustration of AI robot with a “thought bubble” reading its memory notes → solving tasks.

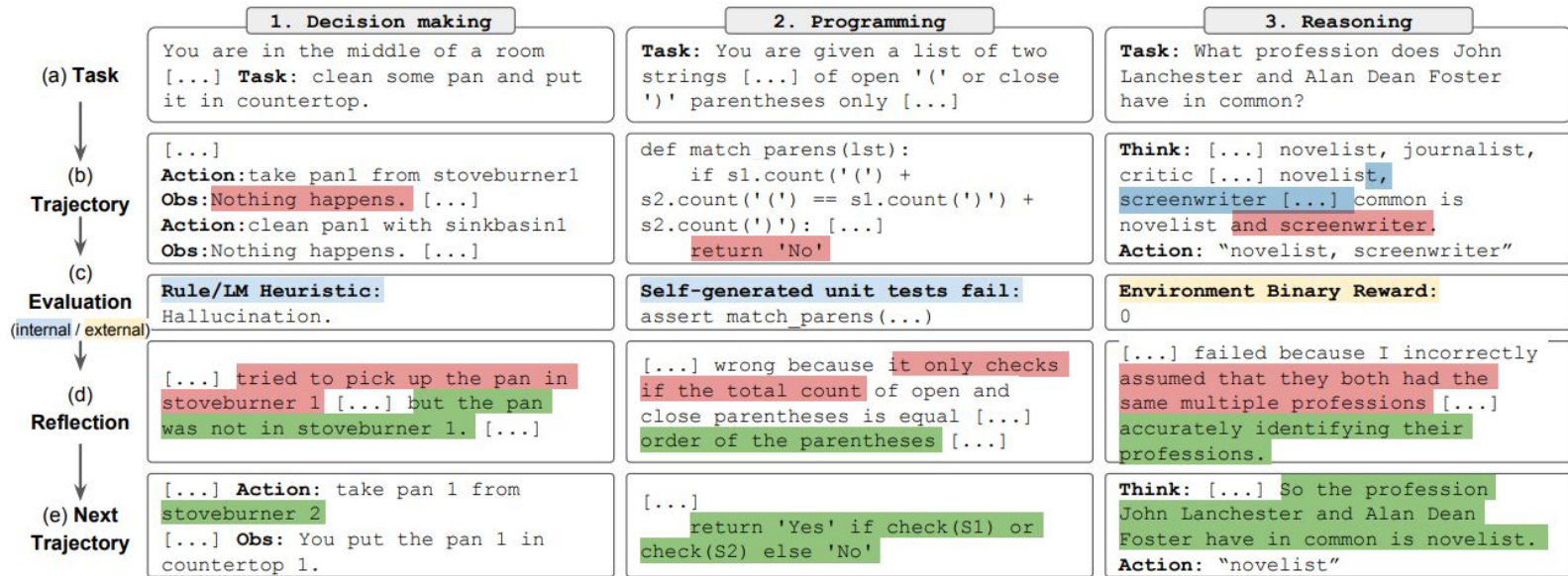
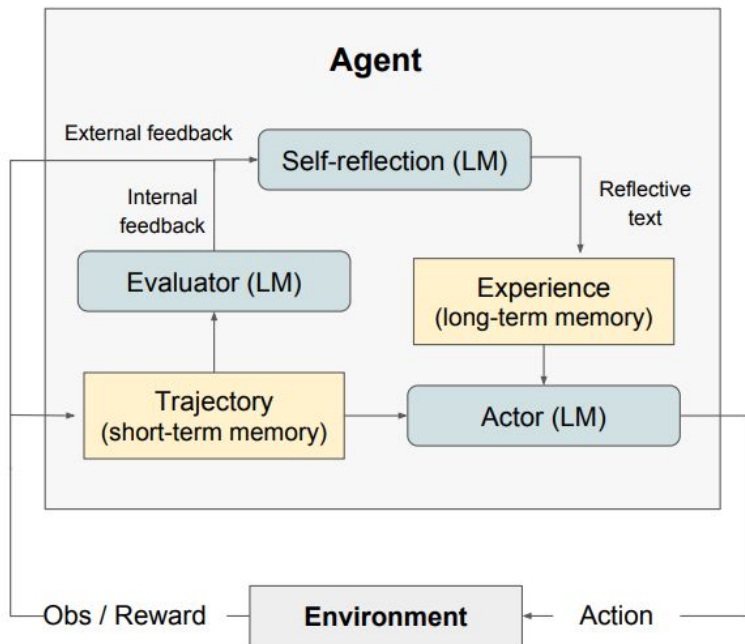


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.



Algorithm 1 Reinforcement via self-reflection

Initialize Actor, Evaluator, Self-Reflection:

M_a, M_e, M_{sr}

Initialize policy $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$

Generate initial trajectory using π_θ

Evaluate τ_0 using M_e

Generate initial self-reflection sr_0 using M_{sr}

Set $mem \leftarrow [sr_0]$

Set $t = 0$

while M_e not pass or $t < \text{max trials}$ **do**

 Generate $\tau_t = [a_0, o_0, \dots, a_i, o_i]$ using π_θ

 Evaluate τ_t using M_e

 Generate self-reflection sr_t using M_{sr}

 Append sr_t to mem

 Increment t

end while

return

Figure 2: (a) Diagram of Reflexion. (b) Reflexion reinforcement algorithm

Prior Work – Reasoning & Decision-Making

How AI Tried to Improve Itself

- AI has tried self-reflection and iterative improvement for reasoning tasks:
 - Self-Refine – Proofreads its own output to make it better (like a student editing an essay). Works only for single answers.
 - Semantic prompt optimization – Tweaks prompts to improve a single answer; also one-shot only.
 - Critic models – Give feedback during multi-step reasoning (like a teaching assistant guiding along the way).
 - Beam search & Decider models – AI explores multiple solutions before picking the best (like thinking ahead in chess).
 - Retry without evaluation – AI tries multiple times but doesn't learn from mistakes.
- Reflexion improves these ideas by creating a persistent memory of reflections, letting the AI learn from past mistakes over time.

Visual Idea:

- Flowchart: Attempt → Feedback → Reflection → Memory → Improved Next Attempt

Prior Work – Programming & Code Improvement

How AI Learned to Code

Main Points:

- Several systems use feedback from tests to improve code:
 - AlphaCode – Tries multiple solutions, checks against hidden tests. Works for simple bugs.
 - CodeT – Generates its own unit tests to score code, but doesn't learn iteratively.
 - Self-Debugging – Fixes existing code using feedback (like a student fixing compiler errors).
 - CodeRL – Uses trial-and-error with rewards to improve coding skills.

Limitations of Previous Methods:

- Many rely on ground truth tests, limiting single-shot success measurement.
- Few methods actually learn from past mistakes to improve future outputs.

Prior Work – Programming & Code Improvement

Key Insight:

- Reflexion bridges the gap: AI remembers errors and self-suggests lessons for better coding over time.

Visual Idea:

- Two columns:
 1. Old Methods: Fix mistakes each time, forget patterns.
 2. Reflexion: Stores reflections → learns and improves for future problems.

Reflexion Advantage Over Prior Work

Why Reflexion Stands Out

Main Points:

- Combines lessons from reasoning and programming AI research.
- Adds persistent self-reflective memory for learning over multiple attempts.
- Works for both reasoning tasks and code generation.
- Mimics human learning: observe mistakes → reflect → improve next attempt.

Analogy:

- Old methods = Student fixes one mistake, forgets it next time.
- Reflexion = Student keeps a reflection diary, improving consistently.

Visual Idea:

- Diagram:
Past Approach: Task → Mistake → Fix → Forget → Next Task
Reflexion Approach: Task → Mistake → Reflection → Memory → Next Task → Success

Reflexion

Teaching AI to Learn Like a Human

Main Points:

- Reflexion is a framework that helps AI improve through reflection, not retraining.
- It splits learning into three main roles:
 1. Actor – tries tasks and generates outputs.
 2. Evaluator – scores the outputs.
 3. Self-Reflection model – turns scores into verbal feedback for learning.
- Analogy: Like a student: tries → gets graded → writes notes to improve next time.

Visual Idea:

- Simple diagram: Actor → Evaluator → Self-Reflection → Memory → Actor loop

Actor – The Task Performer

Actor: Making Decisions and Taking Actions

Main Points:

- Built on a large language model (LLM).
- Observes the environment → generates actions or outputs.
- Uses short-term memory to track current steps and long-term memory for past lessons.
- Can explore different generation strategies (e.g., Chain of Thought, ReAct) to solve tasks.
- Analogy: Like a student writing an answer while remembering what they just did and lessons from previous tests.

Visual Idea:

- Actor box with arrows pointing to: Environment (inputs) and Memory (context).

Evaluator – The Teacher or Grader

Evaluator: Scoring the Actor's Actions

Main Points:

- Judges the Actor's outputs and assigns a score.
- Different scoring strategies for different tasks:
 - Reasoning tasks: check for exact match.
 - Decision-making tasks: use heuristic rules.
 - Programming tasks: optionally use another LLM to evaluate code.
- Scalar rewards alone are often not enough to guide improvement.
- Analogy: Like a teacher giving a grade, but you need comments to know what to improve.

Visual Idea:

- Actor → Evaluator → Score (numeric)
- Thought bubble: "This is correct/incorrect, but why?"

Self-Reflection Model – Learning From Mistakes

Self-Reflection: Turning Feedback into Lessons

Main Points:

- Converts scores and trajectories into verbal feedback.
- Example: “Action A caused mistakes in B and C; next time take A’ instead.”
- Feedback is stored in long-term memory for future tasks.
- Enables the Actor to adapt and improve iteratively.
- Analogy: Student not only sees a grade but writes notes on what went wrong for next time.

Visual Idea:

- Score → Self-Reflection → Memory → Actor reads feedback next round

Memory and Iterative Learning

Memory: Short-Term & Long-Term Lessons

Main Points:

- Short-term memory: recent trajectory (what just happened).
- Long-term memory: distilled feedback from past reflections.
- Actor uses both to make better decisions next time.
- Reflexion works in loops:
 1. Actor generates trajectory.
 2. Evaluator scores it.
 3. Self-Reflection creates feedback.
 4. Feedback stored in memory → next trial improves.
- Usually stores 1–3 reflections to respect LLM context limits.
- Analogy: Like a student reviewing both recent mistakes and long-term lessons while tackling a new problem.

Visual Idea:

- Loop diagram: Trial 1 → Feedback → Memory → Trial 2 → Improved Action → Success

Experiments Overview

Testing Reflexion Across Different Tasks

- Reflexion was tested on three types of tasks:
 - Decision-making – multi-step tasks in virtual environments (AlfWorld).
 - Reasoning – answering complex questions using multiple documents (HotPotQA).
 - Programming – code generation in Python and Rust (HumanEval, MBPP, LeetcodeHardGym).
- Reflexion consistently improved performance over strong baselines:
 - AlfWorld: +22%
 - HotPotQA: +20%
 - HumanEval: +11%
- Analogy: Like a student practicing household chores, logic puzzles, and coding exercises, improving each time by learning from mistakes.

Visual Idea:

- Table or diagram showing Tasks → Reflexion → Improvement %

Decision-Making – AlfWorld

Reflexion in Multi-Step Tasks

- AlfWorld: AI completes tasks like finding objects, moving items, or manipulating tools.
- Baseline issues: AI makes early mistakes (e.g., thinks it picked up a spatula but didn't) → fails the rest of the task.
- Reflexion advantages:
 - Self-reflection identifies early mistakes.
 - Stores lessons in long-term memory.
 - Uses memory to plan efficient future actions.
- Result: Completed 130/134 tasks, learning steadily over 12 trials, unlike baseline agents that plateaued.

Analogy:

- Like a student learning a complicated chore: if they forget to pick up a tool, they note it down and avoid the same mistake next time.

Visual Idea:

- Flowchart: Task → Mistake → Self-Reflection → Memory → Next Trial → Success

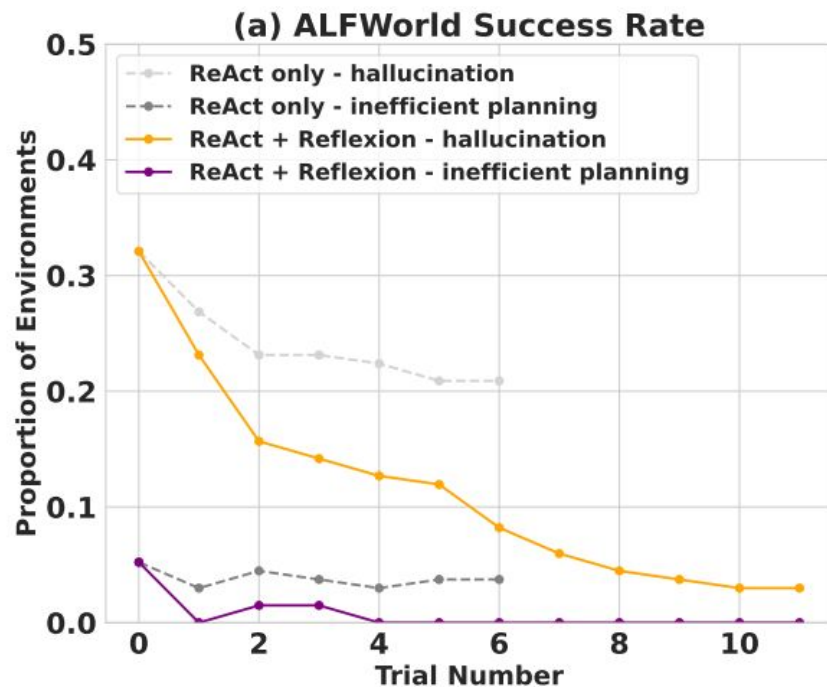
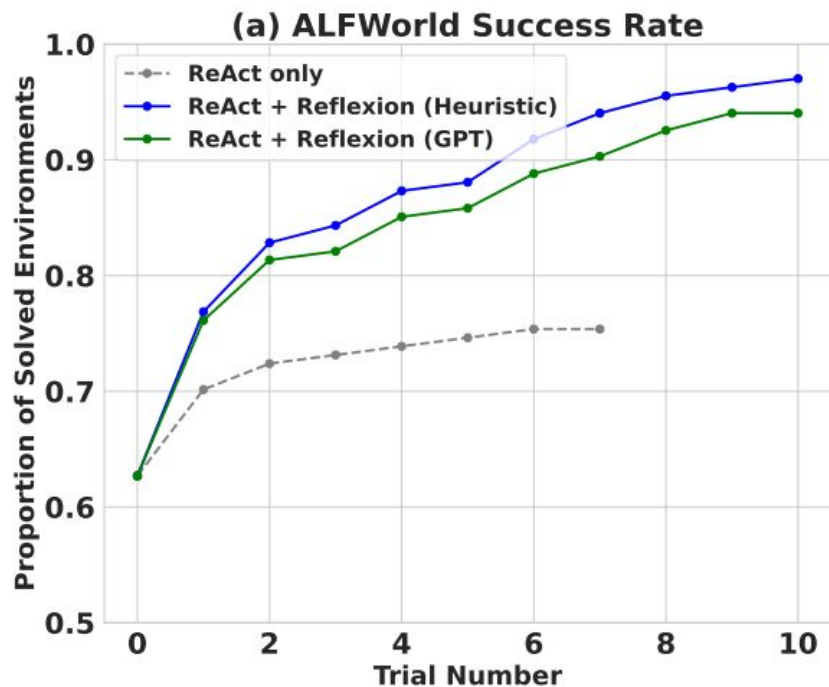


Figure 3: (a) AlfWorld performance across 134 tasks showing cumulative proportions of solved tasks using self-evaluation techniques of (Heuristic) and (GPT) for binary classification. (b) Classification of AlfWorld trajectories by reason of failure.

Reasoning – HotPotQA

Reflexion Improves Step-by-Step Thinking

- HotPotQA: Answering questions using multiple documents.
- Reflexion + Chain-of-Thought (CoT) or ReAct:
 - Step-by-step reasoning
 - Can retry failed questions using self-reflection
- Results:
 - Reflexion improves accuracy by 14% over baselines without ground truth answers.
 - Baselines fail to learn from mistakes; Reflexion learns iteratively.
- Ablation study: Self-reflection + memory > memory alone, proving reflection itself is critical.

Analogy:

- Like a student solving a multi-step puzzle: they write down why their previous answers were wrong, so future attempts are smarter.

Visual Idea:

- Two-column diagram: Baseline: mistake → repeat mistake vs Reflexion: mistake → reflection → correct next attempt

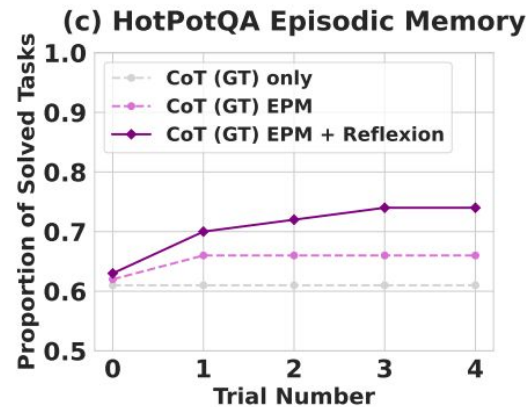
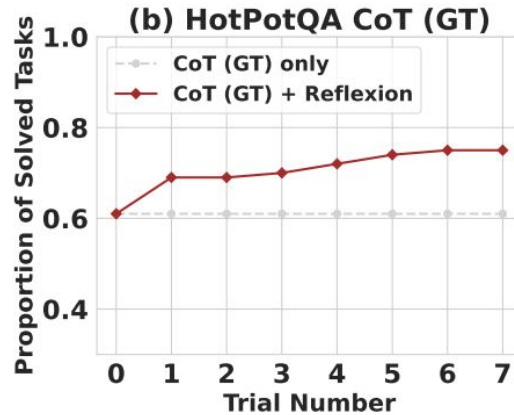
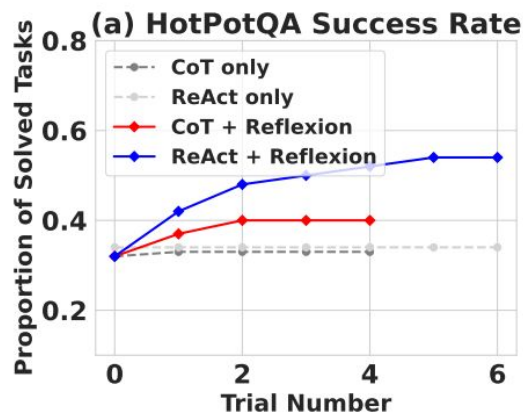


Figure 4: Chain-of-Thought (CoT) and ReAct. Reflexion improves search, information retrieval, and reasoning capabilities on 100 HotPotQA questions. (a) Reflexion ReAct vs Reflexion CoT (b) Reflexion CoT (GT) for reasoning only (c) Reflexion vs episodic memory ablation.

Programming – HumanEval, MBPP, LeetcodeHardGym

Reflexion for Code Generation

- Reflexion generates code and self-tests using unit tests:
 - Writes a few tests
 - Runs them
 - Reflects on failed tests to improve code
- Works for Python and Rust, showing language-agnostic capabilities.
- Results:
 - Achieves 91% pass@1 on HumanEval Python
 - Outperforms baselines and sets state-of-the-art on most benchmarks
- Key insight: both self-reflection and test generation are required; omitting either reduces performance.

Analogy:

- Like a student writing code, testing it, and taking notes on why errors happened to fix future programs more efficiently.

Visual Idea:

- Flow: Write code → Run tests → Failed? → Reflect → Fix code → Store lesson → Next code

Benchmark + Language	Prev SOTA Pass@1	SOTA Pass@1	Reflexion Pass@1
HumanEval (PY)	65.8 (CodeT [5] + GPT-3.5)	80.1 (GPT-4)	91.0
HumanEval (RS)	–	60.0 (GPT-4)	68.0
MBPP (PY)	67.7 (CodeT [5] + Codex [6])	80.1 (GPT-4)	77.1
MBPP (RS)	–	70.9 (GPT-4)	75.4
Leetcode Hard (PY)	–	7.5 (GPT-4)	15.0

Table 1: Pass@1 accuracy for various model-strategy-language combinations. The base strategy is a single code generation sample. All instruction-based models follow zero-shot code generation.

Benchmark + Language	Base	Reflexion	TP	FN	FP	TN
HumanEval (PY)	0.80	0.91	0.99	0.40	0.01	0.60
MBPP (PY)	0.80	0.77	0.84	0.59	0.16	0.41
HumanEval (RS)	0.60	0.68	0.87	0.37	0.13	0.63
MBPP (RS)	0.71	0.75	0.84	0.51	0.16	0.49

Table 2: Overall accuracy and test generation performance for HumanEval and MBPP. For Rust, HumanEval is the hardest 50 problems from HumanEval Python translated to Rust with MultiPL-E [4]. TP: unit tests pass, solution pass; FN: unit tests fail, solution pass; FP: unit tests pass, solution fail; TN: unit tests fail, solution fail.

Key Takeaways

Why Reflexion Works

- Reflexion allows AI to learn from experience like humans do:
 1. Act → 2. Get feedback → 3. Reflect in words → 4. Remember lessons → 5. Improve next trial
- Works across decision-making, reasoning, and programming.
- Iterative self-reflection + memory = faster and more robust learning.
- Analogy: Think of Reflexion as a smart student who keeps a notebook of mistakes and tips to improve across all subjects.

Visual Idea:

- Loop diagram showing Trial → Feedback → Reflection → Memory → Next Trial → Success
- Highlight improvement percentages for each task type

Limitations of Reflexion

What Reflexion Can't Do... Yet

- Reflexion optimizes AI decision-making via verbal reflection, but can get stuck in local minima (suboptimal solutions).
- Memory is limited: currently only stores the last 1–3 reflections (sliding window).
- Potential future improvement: use vector embeddings or databases to expand memory.
- Programming challenges:
 - Non-deterministic or impure functions (outputs vary unexpectedly)
 - Functions affected by hardware or parallel processes
 - Hard to fully test in all scenarios

Analogy:

- Like a student who can only remember the last few mistakes and struggles with unpredictable assignments.

Visual Idea:

- Graphic showing Limited Memory → Reflection → Improvement, with warning signs for unpredictable tasks.

Broader Impact

Potential and Risks of Reflexion

Main Points:

- Reflexion can empower AI agents in automation, robotics, coding, and human interaction.
- Risk: More capable AI can be misused if not carefully monitored.
- Advantage over traditional RL: verbal self-reflection increases interpretability.
 - Humans can check why AI made a decision by reading its reflections.
- Example: AI using a tool in a complex environment can explain its intentions, ensuring safer use.

Analogy:

- Like a student writing notes about why they chose each step, so teachers can understand their reasoning.

Visual Idea:

- Two columns: Benefits vs Risks, with an arrow pointing to Self-Reflection → Transparency

Conclusion

Reflexion in a Nutshell

- Reflexion teaches AI to learn from mistakes using verbal feedback.
- Outperforms current approaches in:
 - Decision-making
 - Reasoning
 - Programming/code generation
- Future directions:
 - More advanced RL techniques like value learning in natural language
 - Off-policy exploration for smarter decision-making
- Reflexion makes AI more interpretable, which is important for safe deployment.

Q&A